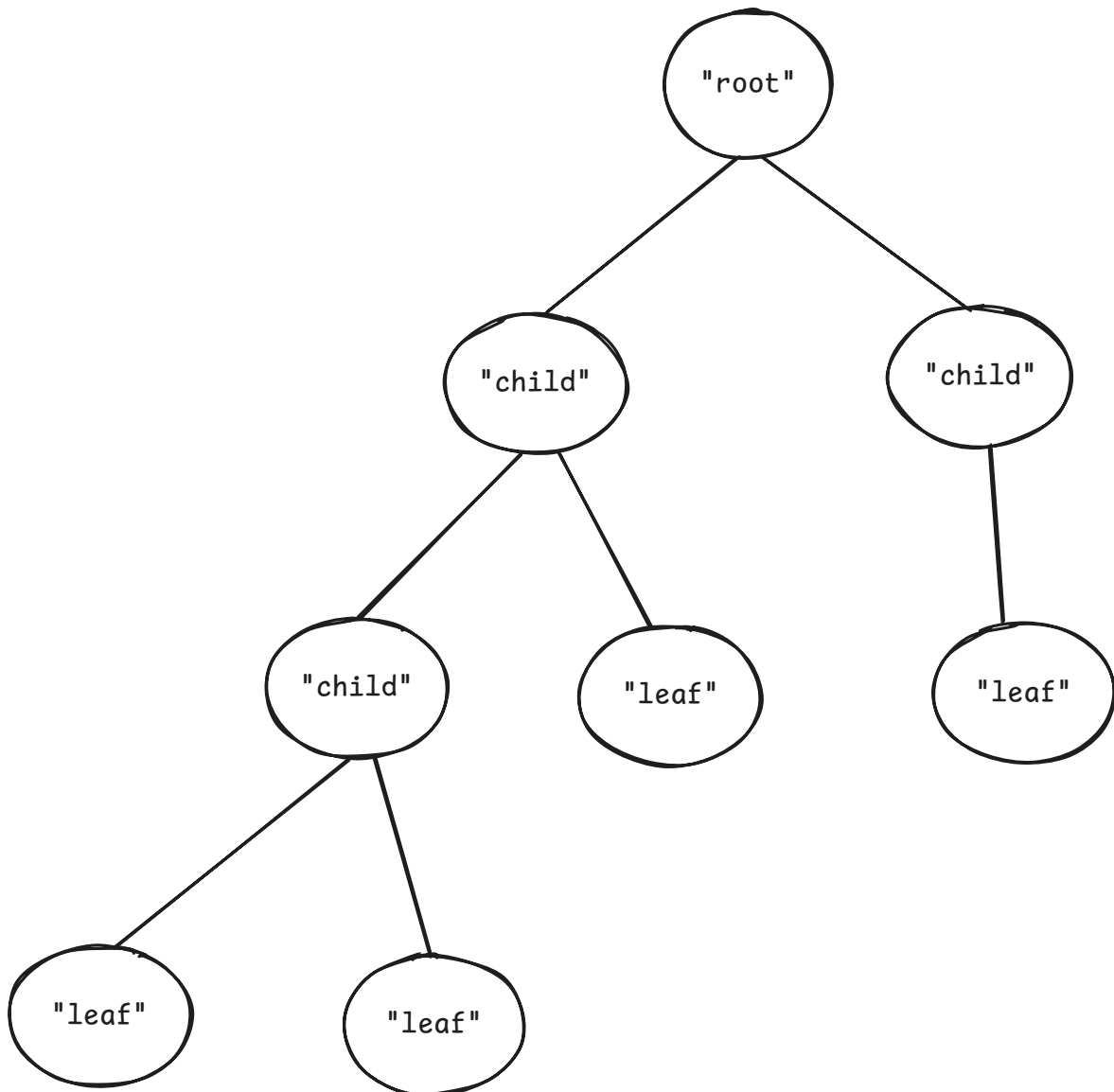


PSET4

Parte A: Recursive Operations on Trees

Un árbol es una estructura de datos jerárquica compuesta por nodos enlazados. El nodo más alto se denomina raíz y tiene ramificaciones que lo enlazan con otros nodos, que a su vez son raíces de sus respectivos subárboles. A continuación se muestra un árbol simple:

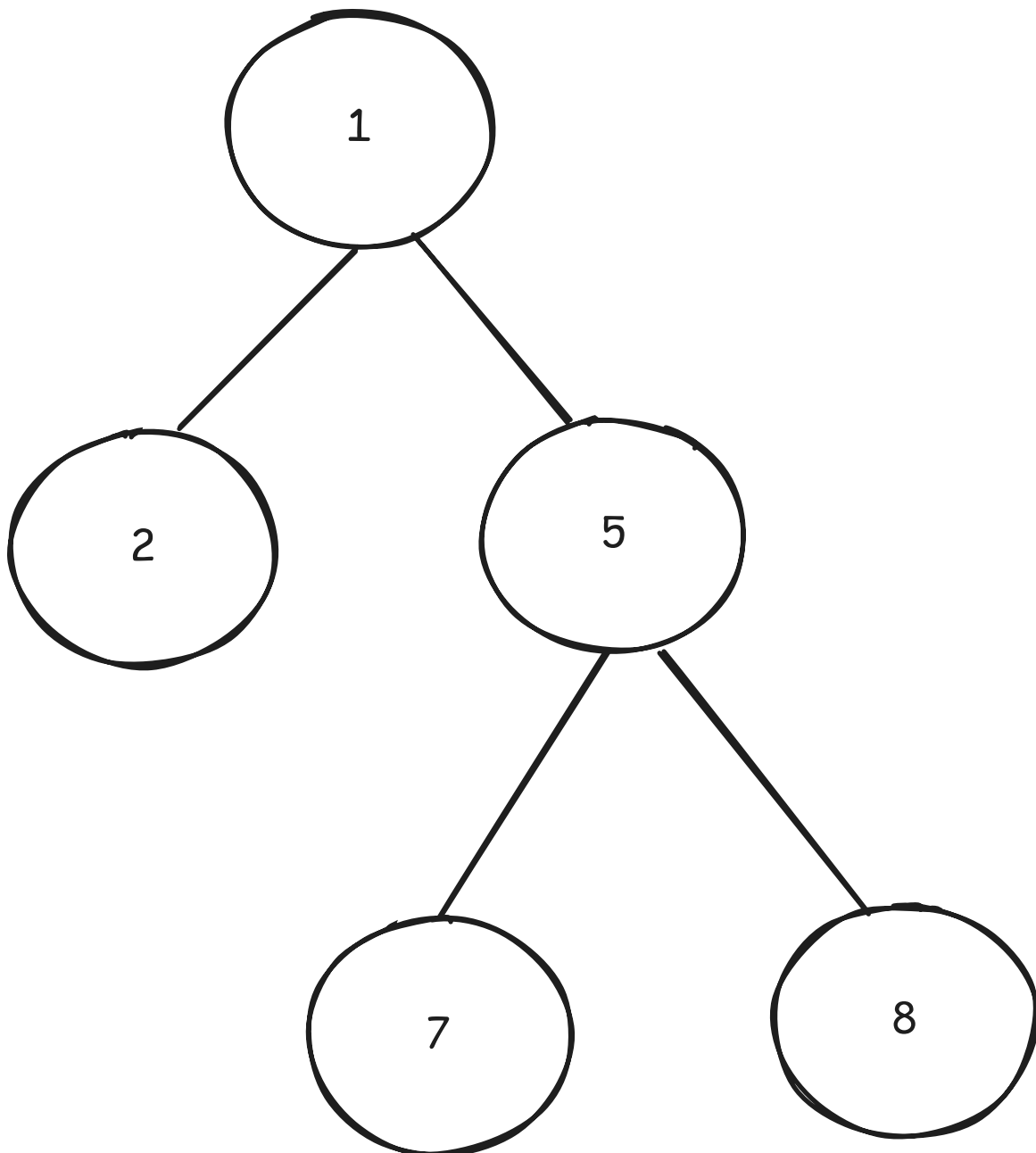


- Cada nodo puede contener datos: en este ejemplo, contienen una cadena con su tipo de nodo.
- Los datos son jerárquicos: cada nodo tiene un padre (excepto la raíz), cada nodo que no es hoja tiene uno o más hijos, y cada nodo hoja tiene cero hijos. Utilizaremos esta nomenclatura en el resto del conjunto de problemas.

- Los árboles son intrínsecamente recursivos: los nodos hijos de una raíz son «raíces» de otros árboles más pequeños (denominados subárboles).

Data Representation Practice

En este conjunto de problemas, utilizaremos un objeto `Node` proporcionado en `tree.py` para representar árboles.



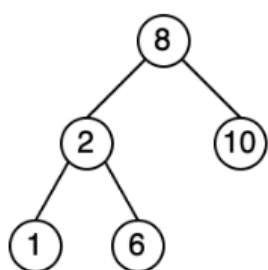
El árbol simple anterior se puede inicializar con el objeto `Node` de la siguiente manera:

```
1 example_tree = Node(1, Node(2), Node(5, Node(7), Node(8)))
```

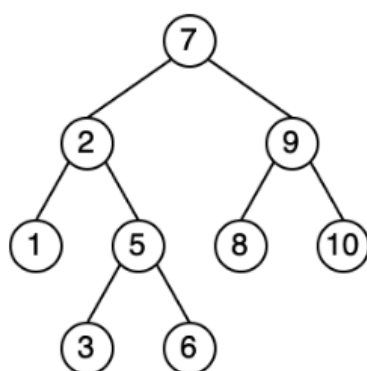
A continuación se ofrece una breve explicación de la clase `Node`:

- Se puede inicializar un nodo con lo siguiente: `Node(value, left_child, right_child)`. `value` contiene el valor almacenado en el nodo, `left_child` contiene opcionalmente el nodo que construye el subárbol izquierdo, e `right_child` hace lo mismo para el subárbol derecho. Si no hay un subárbol, no introduzca ese parámetro o pase `None`.
- Puede obtener el objeto `Node` que contiene los subárboles izquierdo o derecho con `get_left_child()` o `get_right_child()`, respectivamente. Si no hay ningún hijo, esta función devuelve `None`.
- Puede obtener el valor contenido en un `Node` con `get_value()`.

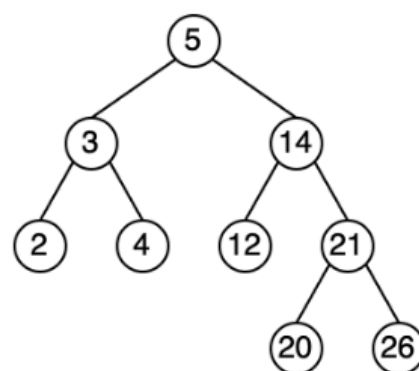
En esta parte practicaremos la inicialización de árboles. Para los árboles que se muestran a continuación, cree objetos que representen con precisión los datos. Colóquelos en las variables en la parte superior de `ps4a.py`, llamadas `tree1`, `tree2` y `tree3`.



Tree 1



Tree 2



Tree 3

Mi solución

```
1 tree1 = Node(8, Node(2, Node(1), Node(6)), Node(10))
2 tree2 = Node(7, Node(2, Node(1), Node(5, Node(3), Node(6))), Node(9,
3 tree3 = Node(5, Node(3, Node(2), Node(4)), Node(14, Node(12), Node(21,
Node(20), Node(26))))
```

Determining The Height of a Tree

La altura de un árbol es el número de aristas entre la raíz y la hoja más alejada. Por ejemplo, en los árboles que has inicializado anteriormente, el árbol 1 tiene una altura de 2 y los árboles 2 y 3 tienen una altura de 3. Escribe una función recursiva,

`find_tree_height`, que determine la profundidad de un árbol. Esta función debe ser recursiva; las implementaciones no recursivas recibirán un cero.

Pista

Dado un árbol `T`:

- Caso base: si `T` es una hoja (leaf), entonces tendrá altura 0.
- Caso recursivo: Recursivamente encuentra la altura de `T`, los subárboles izquierda y derecha, y toma el máximo de los dos. Añade 1 al máximo y retorna ese valor.

Mi solución

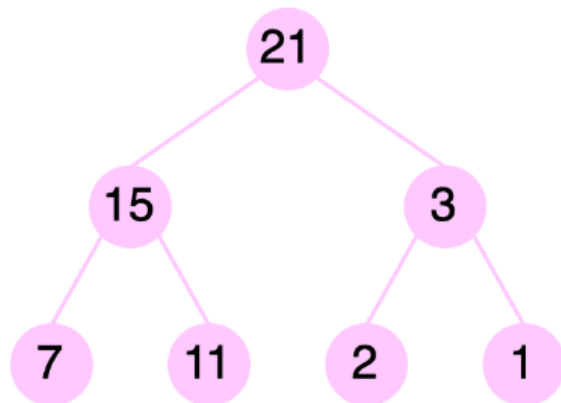
```
1  def find_tree_height(tree):
2      '''
3      Find the height of the given tree
4      Input:
5          tree: An element of type Node constructing a tree
6      Output:
7          The integer depth of the tree
8      '''
9      if tree.get_left_child() is None and tree.get_right_child() is None:
10         return 0
11
12     left_height = 0
13     right_height = 0
14
15     if tree.get_left_child() is not None:
16         left_height = find_tree_height(tree.get_left_child())
17     if tree.get_right_child() is not None:
18         right_height = find_tree_height(tree.get_right_child())
19
20     return 1 + max(left_height, right_height)
```

Heaps

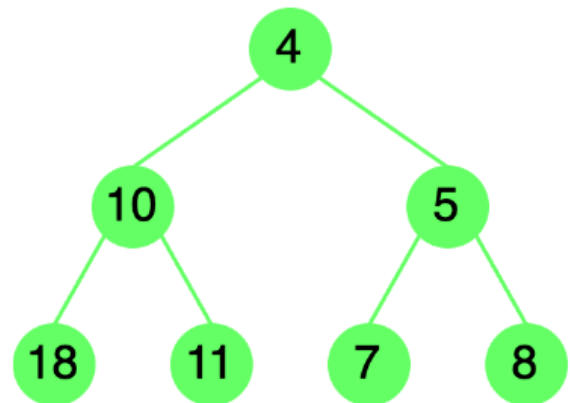
Un tipo especial de árboles son los montones. Hay dos tipos de montones: montones máximos y montones mínimos.

En un montón máximo, para cada nodo `N`, `N` es el valor máximo en el árbol con raíz en `N`. Esto significa que todos los elementos almacenados en los subárboles izquierdo y derecho de `N` son menores que el valor almacenado en `N`.

Por el contrario, en un montón mínimo, N es el valor mínimo en el árbol con raíz en N , por lo que todos los elementos N almacenados en los subárboles izquierdo y derecho, son mayores que el valor almacenado en N .



Max Heap



Min Heap

Escribe la función `is_heap` para determinar rápidamente si un árbol es un montón máximo o mínimo, dependiendo del parámetro `compare_func`. La función `compare_func` toma dos argumentos, `child_value` y `parent_value`.

Para los montones máximos, esta función devolverá `True` si `child_value < parent_value` y `False` en caso contrario. Para los montones mínimos, la función devolverá `True` si `child_value > parent_value` y `False` en caso contrario. Conceptualmente, esto te permite escribir una función que pueda determinar tanto los montones máximos como los mínimos basándose en un parámetro, en lugar de escribir dos métodos separados con código muy similar.

A continuación, te proporcionamos implementaciones funcionales de `compare_func` para un montón máximo y uno mínimo, respectivamente.

```
1  # max heap comparator
2  def compare_func(child_value, parent_value):
3      if child_value < parent_value:
4          return True
5      return False
6
7  # min heap comparator
8  def compare_func(child_value, parent_value):
9      if child_value > parent_value:
10         return True
11     return False
```

- **Caso base.** Si T es una hoja, entonces es un heap (tanto para heaps máximos como mínimos)
- **Caso recursivo.** Para un nodo T, verificar recursivamente si los subárboles izquierdo y derecho de T también son heaps y que T es el elemento máximo o mínimo (usando la función de comparación `compare_func` pasada a `is_heap`) de su subárbol. Si todas las condiciones anteriores son verdaderas, entonces el árbol con raíz en T es un heap. De lo contrario, no lo es.

Mi solución

```

1  def is_heap(tree, compare_func):
2      '''
3          Determines if the tree is a max or min heap depending on compare_func
4          Inputs:
5              tree: An element of type Node constructing a tree
6              compare_func: a function that compares the child node value to the
parent node value
7                  i.e. op(child_value,parent_value) for a max heap would return
True if child_value < parent_value and False otherwise
8                  op(child_value,parent_value) for a min meap would return
True if child_value > parent_value and False otherwise
9          Output:
10             True if the entire tree satisfies the compare_func function; False
otherwise
11         '''
12         if find_tree_height(tree) == 0:
13             return True
14
15         if tree.get_left_child() is not None:
16             left = tree.get_left_child()
17             if not compare_func(left.get_value(), tree.get_value()):
18                 return False
19             if not is_heap(left, compare_func):
20                 return False
21
22         if tree.get_right_child() is not None:
23             right = tree.get_right_child()
24             if not compare_func(right.get_value(), tree.get_value()):
25                 return False
26             if not is_heap(right, compare_func):
27                 return False
28
29         return True

```

Parte B: Encryption with One Time Pads

Introducción

En este problema implementaremos una técnica de encriptación simple que, cuando se implementa correctamente, no puede ser quebrada.

Vocabulario importante:

- **encriptación**: el proceso de oscurecer o codificar mensajes para hacerlos ilegibles
- **desencriptación**: el proceso de convertir mensajes encriptados de vuelta a su forma original y legible
- **texto plano**: el mensaje original y legible
- **texto cifrado**: el mensaje encriptado. Un texto cifrado aún contiene toda la información del mensaje original, aunque parezca galimatías

Cómo Funciona un Pad de Un Solo Uso

La idea de un pad de un solo uso es "desplazar" cada carácter en tu mensaje de texto plano por una cantidad aleatoria. Esto resulta en un texto cifrado que no puede ser desencriptado sin la lista de cantidades aleatorias por las que tus letras fueron desplazadas, llamada el pad.

Conceptos clave del desplazamiento:

- Todos los caracteres se representan como números en ASCII
- Solo nos interesan los caracteres con valores ASCII de 32 a 126 (95 caracteres en total)
- Cuando el desplazamiento "se envuelve", el último carácter  (126) desplazado por 1 sería un espacio (32)

Ejemplos de desplazamiento:

- "A" (65) + 5 = "F" (70)
- " " (32) - 1 = "~" (126)
- "}" (125) + 4 = "'" (34)

Usando Clases y Herencia

Este problema introduce el concepto de **clases y herencia** con:

- **Clase padre:** `Message` - contiene métodos que tanto mensajes de texto plano como encriptados necesitan
- **Clases hijas:**
 - `PlaintextMessage` - métodos específicos para mensajes de texto plano
 - `EncryptedMessage` - métodos específicos para textos cifrados

Clase Message

Métodos a implementar:

1. `__init__(self, input_text)` - Constructor
2. `get_text(self)` - Obtener el texto del mensaje
3. `shift_char(self, char, shift)` - Desplazar un carácter por una cantidad específica
4. `apply_pad(self, pad)` - Aplicar un pad completo al mensaje

Mi solución

```

1  class Message(object):
2      def __init__(self, input_text):
3          '''
4              Initializes a Message object
5
6              input_text (string): the message's text
7
8              a Message object has one attribute:
9                  the message text
10             '''
11             self.text_message = input_text
12
13         def __repr__(self):
14             '''
15                 Returns a human readable representation of the object
16                 DO NOT CHANGE
17
18                 Returns: (string) A representation of the object
19             '''
20             return f'''Message('{self.get_text()}')'''
21
22         def get_text(self):
23             '''
24                 Used to access the message text outside of the class
25
26                 Returns: (string) the message text
27             '''
28             return self.text_message
29

```



```

30     def shift_char(self, char, shift):
31         '''
32         Used to shift a character as described in the pset handout
33
34         char (string): the single character to shift.
35             ASCII value in the range: 32<=ord(char)<=126
36         shift (int): the amount to shift char by
37
38         Returns: (string) the shifted character with ASCII value in the
range [32, 126]
39         '''
40         num = ((ord(char) + shift) - 32) % 95 + 32
41         return chr(num)
42
43     def apply_pad(self, pad):
44         '''
45         Used to calculate the ciphertext produced by applying a one time
pad to the message text.
46         For each character in the text at index i shift that character by
47         the amount specified by pad[i]
48
49         pad (list of ints): a list of integers used to encrypt the message
text
50
51             len(pad) == len(the message text)
52
53         Returns: (string) The ciphertext produced using the one time pad
54         '''
55         result = ''
56         for l,m in zip(pad, self.text_message):
57             result += self.shift_char(m, l)
58         return result

```

Clase PlaintextMessage

Métodos a implementar:

1. `__init__(self, input_text, pad=None)` - Constructor con pad opcional
2. `generate_pad(self)` - Generar un pad aleatorio
3. `get_pad(self)` - Obtener una copia del pad
4. `get_ciphertext(self)` - Obtener el texto cifrado
5. `change_pad(self, new_pad)` - Cambiar el pad usado

```

1  class PlaintextMessage(Message):
2      def __init__(self, input_text, pad=None):
3          '''
4          Initializes a PlaintextMessage object.
5
6          input_text (string): the message's text
7          pad (list of ints OR None): the pad to encrypt the input_text or
None if left empty
8          if pad is not None then len(pad) == len(self.input_text)

```

```

9
10     A PlaintextMessage object inherits from Message. It has three
attributes:
11         the message text
12         the pad (list of integers, determined by pad
13         or generated randomly using self.generate_pad() if pad is
None)
14         the ciphertext (string, input_text encrypted using the pad)
15     '''
16     super().__init__(input_text)
17
18     if pad is None:
19         self.pad = self.generate_pad()
20     else:
21         self.pad = pad.copy()
22
23     self.ciphertext = self.apply_pad(self.pad)
24
25     def __repr__(self):
26         '''
27         Returns a human readable representation of the object
28         DO NOT CHANGE
29
30         Returns: (string) A representation of the object
31         '''
32         return f'''PlaintextMessage('{self.get_text()}',
{self.get_pad()})'''
33
34     def generate_pad(self):
35         '''
36         Generates a one time pad which can be used to encrypt the message
text.
37
38         The pad should be generated by making a new list and for each
character
39             in the message choosing a random number in the range [0, 110)
and
40             adding that number to the list.
41
42         Returns: (list of integers) the new one time pad
43         '''
44         text_length = len(self.get_text())
45         pad = []
46         for i in range(text_length):
47             pad.append(random.randint(0, 109))
48         return pad
49
50     def get_pad(self):
51         '''
52         Used to safely access your one time pad outside of the class
53
54         Returns: (list of integers) a COPY of your pad
55         '''
56         return self.pad.copy()
57
58     def get_ciphertext(self):

```

```

59         '''
60         Used to access the ciphertext produced by applying pad to the
message text
61
62         Returns: (string) the ciphertext
63         '''
64         return self.apply_pad(self.pad)
65
66
67     def change_pad(self, new_pad):
68         '''
69         Changes the pad used to encrypt the message text and updates any
other
70         attributes that are determined by the pad.
71
72         new_pad (list of ints): the new one time pad that should be
associated with this message.
73         len(new_pad) == len(the message text)
74
75         Returns: nothing
76         '''
77         self.pad = new_pad
78         self.ciphertext = self.apply_pad(self.pad)

```

Clase EncryptedMessage

Concepto de descriptación: Para descriptar, simplemente aplicamos el pad negativo. Si encriptamos con `[5, 1, 7, 2]`, descriptamos con `[-5, -1, -7, -2]`.

Métodos a implementar:

1. `__init__(input_text)` - Constructor
2. `decrypt_message(self, pad)` - Descriptar usando un pad dado

Mi solución

```

1  class EncryptedMessage(Message):
2      def __init__(self, input_text):
3          '''
4          Initializes an EncryptedMessage object
5
6          input_text (string): the ciphertext of the message
7
8          an EncryptedMessage object inherits from Message. It has one
attribute:
9              the message text (ciphertext)
10         '''
11         super().__init__(input_text)
12

```

```

13     def __repr__(self):
14         '''
15         Returns a human readable representation of the object
16         DO NOT CHANGE
17
18         Returns: (string) A representation of the object
19         '''
20         return f'''EncryptedMessage('{self.get_text()}')'''
21
22     def decrypt_message(self, pad):
23         '''
24         Decrypts the message text that was encrypted with pad as described
25         in the writeup
26
27         pad (list of ints): the new one time pad used to encrypt the
28         message.
29
30         len(pad) == len(the message text)
31
32         Returns: (PlaintextMessage) the decrypted message (containing the
33         pad)
34         '''
35         inverse_pad = [-x for x in pad]
36         decrypted_text = self.apply_pad(inverse_pad)
37         return PlaintextMessage(decrypted_text, pad)

```

Parte C: Using Your Classes

Ahora que ya creaste tus clases en `ps4b.py`, vamos a aprender cómo usarlas en `ps4c.py`. Al principio de ese archivo vas a ver esta línea:

```
1 import ps4b # Importa tu trabajo de la Parte B
```

Esto importa el código que escribiste en `ps4b.py`, siempre y cuando esté en la **misma carpeta** que `ps4c.py`.

Para usar una clase que creaste en la Parte B, podés inicializarla así:

```
1 my_message = ps4b.Message("¡Mi mensaje!")
```

Funciones auxiliares

Hay algunas funciones ya implementadas para vos:

- `load_words()`: carga una lista de palabras en inglés.
- `is_word(wordlist, word)`: verifica si `word` es una palabra válida del diccionario.
- `get_story_string()`: devuelve un string cifrado que representa una historia.

Estas funciones te van a servir para implementar

`decrypt_message_try_pads()` y `decode_story()`.

No necesitás entender **cómo** funcionan internamente, pero sí **qué hacen** y cómo usarlas. Leé sus docstrings para saberlo.



Descifrando mensajes cifrados

Un "one time pad" es muy seguro. Si no sabés cuál se usó, no podés descifrar el mensaje. Pero si tenés una **lista de pads posibles**, podés probar uno por uno y ver cuál produce una frase que tenga sentido.

La estrategia es:

1. Por cada pad en la lista:
 - Usás ese pad para **descifrar el mensaje**.
 - Separás el mensaje en palabras (con `.split()`).
 - Contás cuántas de esas palabras son **palabras válidas en inglés**, usando `is_word()`.
2. El pad que produzca la **mayor cantidad de palabras válidas** es probablemente el correcto.
3. Si hay empate entre dos pads, devolvés el **último** que tuvo la mayor cantidad de palabras válidas.

Implementar el método:

```
1 decrypt_message_try_pads(self, pads)
```

- `pads` es una lista de pads posibles (cada pad es una lista de enteros).
- Tenés que probar todos.
- Usá `is_word(wordlist, word)` para contar palabras válidas.
- Podés usar `load_words()` para conseguir la lista de palabras en inglés.
- También podés usar el método `.split()` de los strings para separar por espacios.

Mi solución

```
1 def decrypt_message_try_pads(ciphertext, pads):
2     '''
3     Given a string ciphertext and a list of possible pads
4     used to create it find the pad used to create the ciphertext
```

```

5
6     We will consider the pad used to create it the pad which
7     when used to decrypt ciphertext results in a plaintext
8     with the most valid English words. In the event of ties return
9     the last pad that results in the maximum number of valid English
    words.
10
11     ciphertext (EncryptedMessage): The ciphertext
12     pads (list of lists of ints): A list of pads which might have been
    used
13         to encrypt the ciphertext
14
15     Returns: (PlaintextMessage) A message with the decrypted ciphertext
    and the best pad
16     '''
17     wordlist = load_words("words.txt")
18
19     mejor_pad = None
20     mejor_mensaje = None
21     max_validas = -1
22
23     for pad in pads:
24         descifrado = ciphertext.decrypt_message(pad)
25         texto = descifrado.get_text()
26         palabras = texto.split()
27         validas = sum([is_word(wordlist, palabra) for palabra in
    palabras])
28
29         if validas >= max_validas:
30             max_validas = validas
31             mejor_pad = pad
32             mejor_mensaje = descifrado
33
34     return mejor_mensaje

```

Decodificando una historia

Bob está tratando de mandarle una historia secreta a Alice, cifrada con un one time pad. Nosotros **interceptamos la historia cifrada** y también la lista de pads que Bob compartió con Alice. Pero no sabemos cuál de todos usó para cifrar el mensaje.

Tu tarea es descubrir la historia original.

Implementá:

```
1 decode_story()
```

Esta función debe:

- Usar `get_story_string()` para obtener el mensaje cifrado.

- Usar `get_story_pads()` para obtener la lista de pads posibles.
- Usar tu método `decrypt_message_try_pads()` para descifrar el mensaje.
- Devolver el texto descifrado que tiene sentido.

Mi solución

```
1  def decode_story():
2      '''
3      Write your code here to decode Bob's story using a list of possible
4      pads
5      Hint: use the helper functions get_story_string and get_story_pads and
6      your EncryptedMessage class.
7
8      Returns: (string) the decoded story
9      '''
10     texto_cifrado = get_story_string()
11     pads_posibles = get_story_pads()
12     mensaje_cifrado = ps4b.EncryptedMessage(texto_cifrado)
13     mensaje_descifrado = decrypt_message_try_pads(mensaje_cifrado,
14     pads_posibles)
15     return mensaje_descifrado.get_text()
```